

AI-Recipe-Generator

Mancini Marco, Passino Francesco

1. Introduzione

L'obiettivo di questo progetto è realizzare un sistema di *Recipe Retrieval*, cioè un sistema in grado di individuare le ricette più adatte a partire da un insieme di ingredienti forniti dall'utente. Le ricette non vengono generate da zero ma vengono ricercate all'interno di un dataset sfruttando tecniche di *Information Retrieval*.

Durante lo sviluppo del progetto è stato utilizzato il dataset *RecipeNLG* per implementare il sistema di retrieval, basato sul modello *Sentence-BERT* (*SBERT*) e sulla libreria *FAISS*.

E' stata sviluppata anche una semplice interfaccia web con *Flask* che consente all'utente di inserire gli ingredienti che ha a disposizione e ottenere le ricette più indicate.

2. EDA (Exploratory Data Analysis)

2.1 Overview

Dallo studio del dataset sono emersi i seguenti punti:

- dimensione: 2.1 GB
- righe: 2 231 142 (ogni riga rappresenta una ricetta)
- colonne: 7 (ogni colonna descrive una caratteristica della ricetta)

Segue l'elenco delle colonne:

- *Unnamed: 0*, identifica univocamente la ricetta

Tipo	Elementi null	Elementi duplicati
int64	0	0

- *Title*, contiene il titolo della ricetta

Tipo	Elementi null	Elementi duplicati
str	1	918 271

- *Ingredients*, contiene le dosi degli ingredienti

Tipo	Elementi null	Elementi duplicati
str	0	4 780

- *Directions*, contiene le istruzioni della ricetta

Tipo	Elementi null	Elementi duplicati
str	0	19 498

- *Link*, contiene l'URL della ricetta originale

Tipo	Elementi null	Elementi duplicati
str	0	0

- *Source*, contiene la fonte di provenienza della ricetta

Tipo	Elementi null	Elementi duplicati
str	0	2 231 140

- *NER*, contiene la lista degli ingredienti

Tipo	Elementi null	Elementi duplicati
str	0	97 646

2.2 Preprocessing

Il dataset è stato filtrato prendendo solamente le colonne rilevanti, ovvero le colonne *title*, *ingredients*, *directions*, *NER*.

Queste colonne, come mostrato nel paragrafo sopra, sono di tipo stringa e hanno la seguente forma:

```
"['ingrediente_1', 'ingrediente_2']"
```

Ogni colonna è quindi una stringa che rappresenta una lista (fa eccezione *title* che è una semplice stringa). Dopo averle portate tutte in lower case, è stata necessaria una conversione da stringa a lista python.

In questo modo è stato possibile aggiungere delle colonne utili allo studio del dataset, quali:

- *#ingredients*, contiene il numero di ingredienti della ricetta
- *#steps*, contiene il numero di passaggi della ricetta
- *directions_len*, contiene il numero di parole delle istruzioni

Il dataset è stato infine salvato nel formato *parquet* in modo da velocizzare i caricamenti futuri.

2.3 Analisi duplicati

Il dataset presenta degli elementi duplicati che verranno adesso analizzati tramite i seguenti dati:

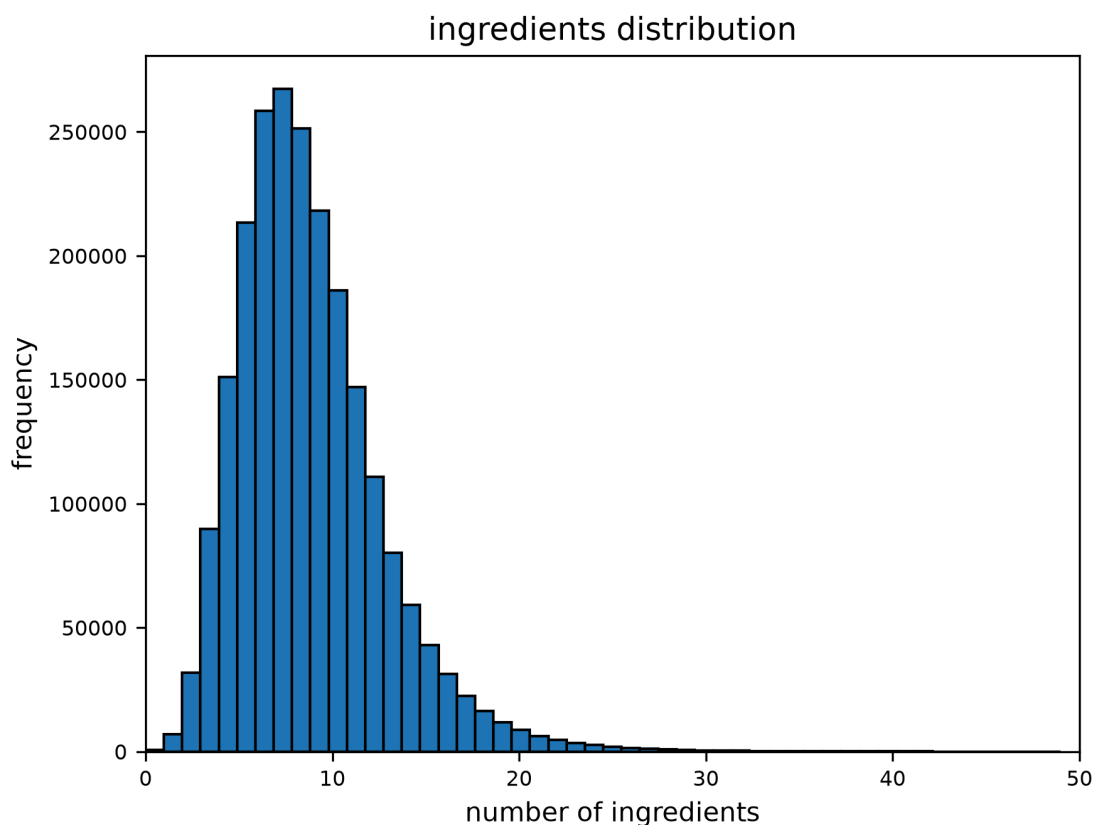
Colonna	Media	Dev.Std	Min	25%	50%	75%	Max
Title	1.699438	13.59383	1	1	1	1	4 099
Ingredients	1.002147	0.06684430	1	1	1	1	28
Directions	1.008816	0.4389496	1	1	1	1	274
NER	1.045768	0.7292958	1	1	1	1	573

Si noti che questi dati sono stati generati prendendo in considerazione l'intera lista di dosi-istruzioni-ingredienti in ogni riga. Stiamo quindi analizzando quante volte quelle stesse liste compaiono all'interno del dataset.

Guardando la tabella notiamo che in media ogni titolo è presente 1.7 volte, ma il 75% delle ricette è presente singolarmente. Questo ci dice che in generale ci sono molte ricette uniche in quanto i titoli duplicati sono sbilanciati verso alcune preparazioni, come "Chicken Casserole" che è presente 4099 volte. Notiamo però anche che le liste di dosi, istruzioni e ingredienti sono quasi sempre uniche. Quindi anche se alcuni titoli sono condivisi, la preparazione di una stessa ricetta può differire a seconda degli ingredienti.

Le osservazioni appena fatte vengono confermate dall'unicità di ogni riga del dataset. Non esistono infatti ricette con stesso titolo, ingredienti, istruzioni e dosi.

2.4 Distribuzione degli ingredienti

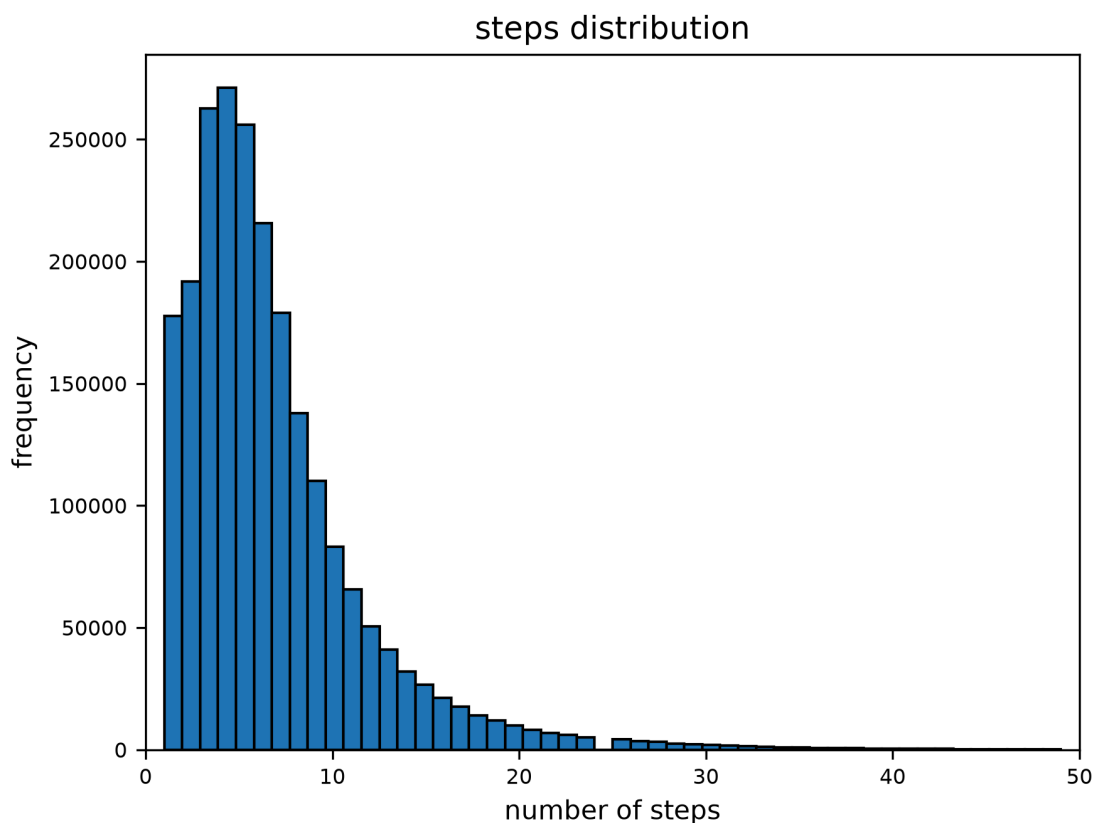


La distribuzione degli ingredienti segue l'andamento di una gaussiana, dove la maggior parte delle ricette contiene fra i 3-13 ingredienti con dei picchi di concentrazione intorno ai 7-9. Questo ci dice che nel dataset sono presenti

principalmente ricette semplici che richiedono un numero limitato di ingredienti. Notiamo anche che ci sono ricette con 0 ingredienti, 573 in totale.

Il grafico viene mostrato per le ricette fino a 50 ingredienti per motivi di leggibilità. Sono tuttavia poche, 58, le ricette che superano questa soglia.

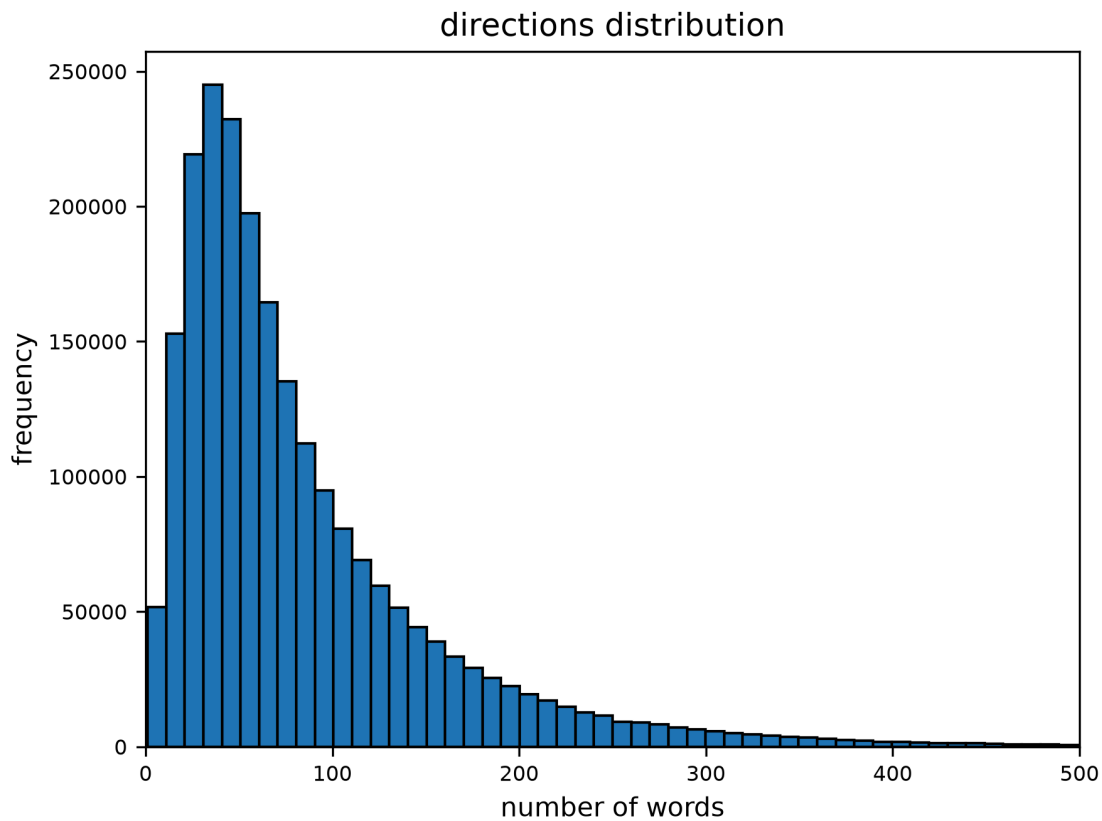
2.5 Distribuzione dei passaggi



La distribuzione dei passaggi segue un andamento simile alla distribuzione degli ingredienti. La maggior parte delle ricette contiene meno di 15 passaggi con dei picchi di concentrazione intorno ai 3-5. Anche in questo caso notiamo che sono presenti molte ricette con pochi passaggi, mentre sono un numero inferiore quelle più complesse.

Il grafico viene mostrato per le ricette fino a 50 step per motivi di leggibilità. Sono tuttavia poche, 1242, le ricette che superano questa soglia.

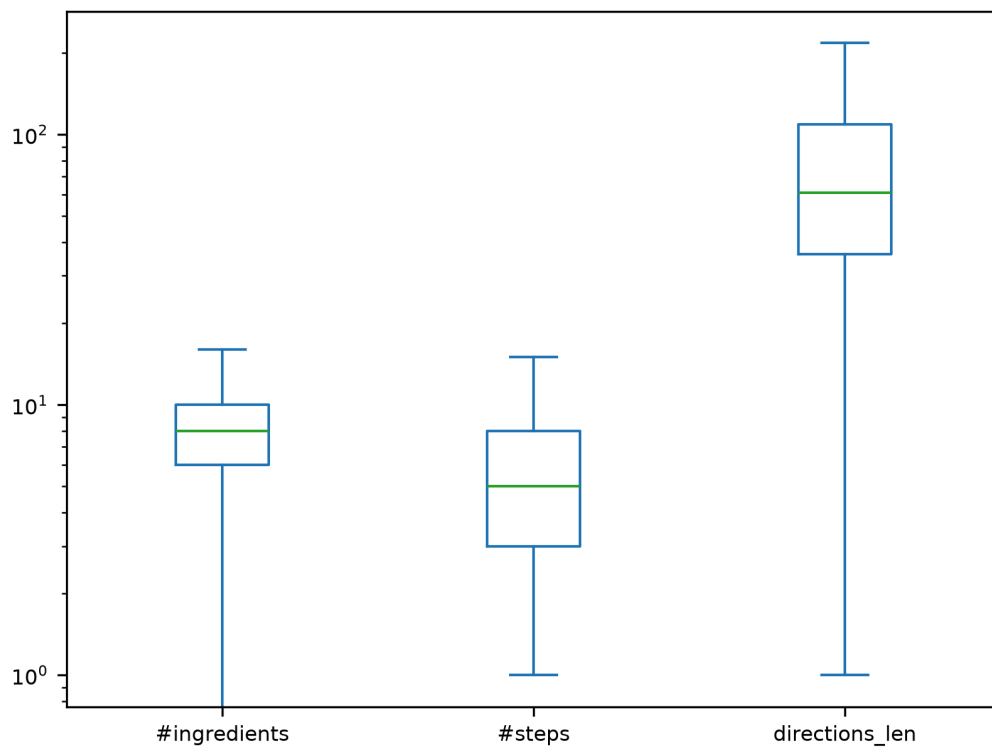
2.6 Distribuzione delle istruzioni



Questa volta notiamo una differenza. La maggior parte delle ricette contiene delle istruzioni lunghe fra le 20-100 parole con dei picchi di concentrazione intorno alle 30-60. Notiamo però che la distribuzione è più dilatata rispetto ai due casi precedenti, quindi il numero di parole nelle istruzioni di una ricetta è più variabile rispetto al numero di ingredienti e il numero di passaggi.

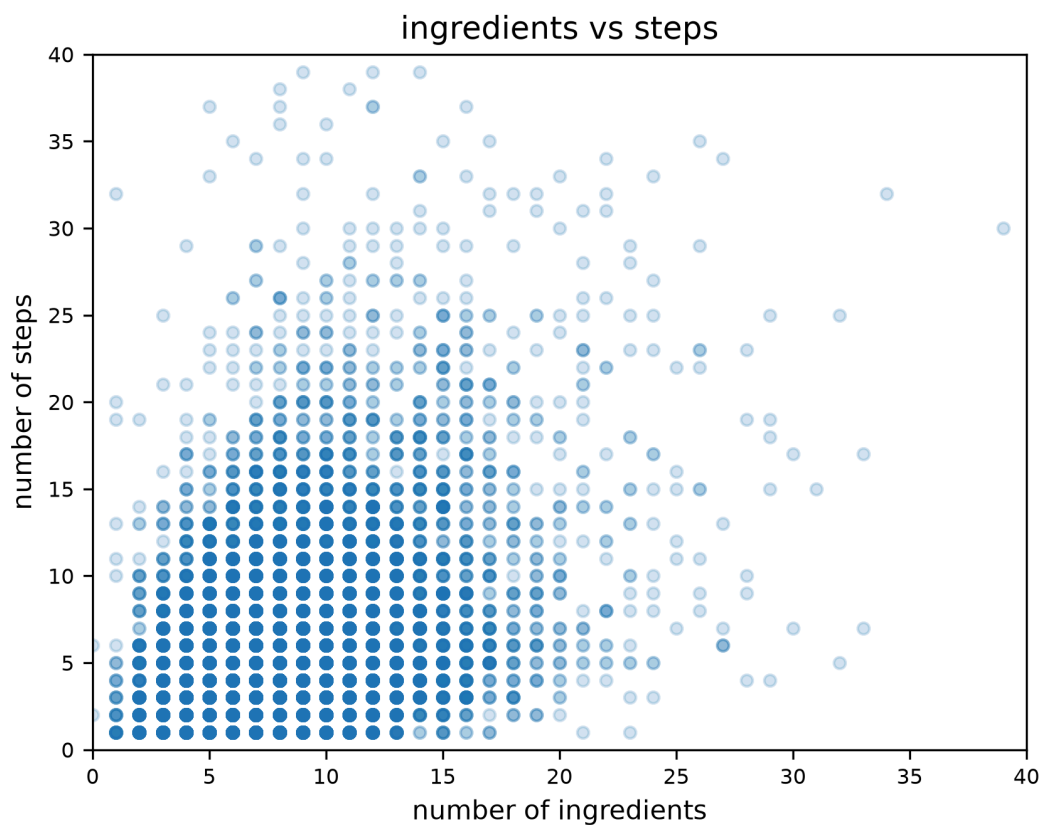
Il grafico viene mostrato per le ricette fino a 500 parole per motivi di leggibilità. Sono tuttavia poche, 8311, le ricette che superano questa soglia.

2.7 Boxplot delle distribuzioni



L'ultima considerazione fatta nel paragrafo precedente viene messa in luce con questo grafico. Il numero di ingredienti e il numero di passaggi è abbastanza compatto, mentre il numero di parole delle istruzioni è più variabile, in quanto una ricetta può essere descritta in maniera sintetica o più dettagliata.

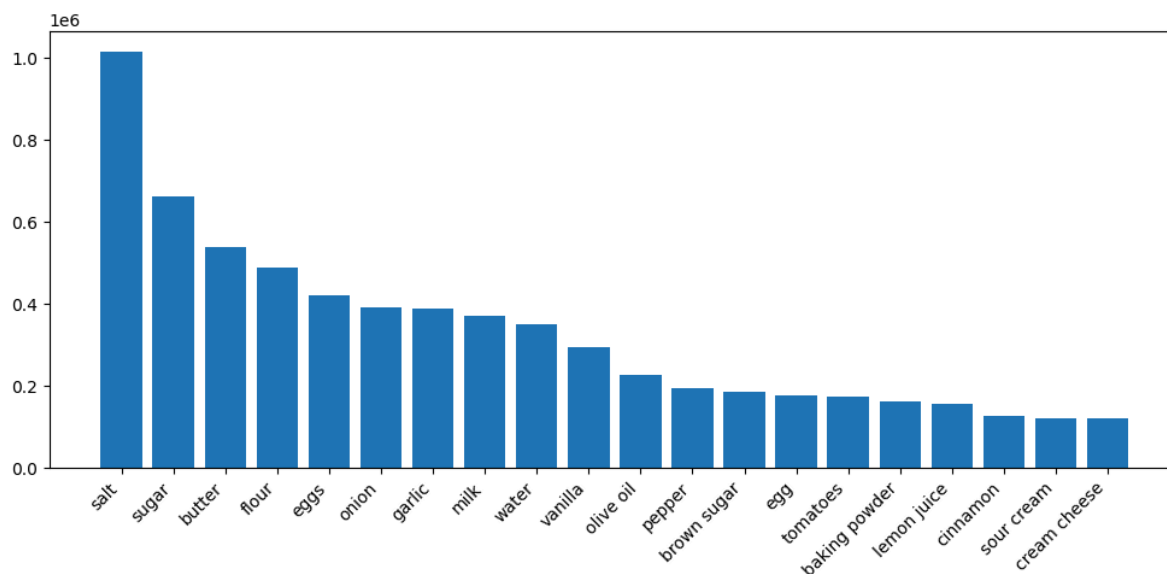
2.8 Relazione tra ingredienti e passaggi



Il grafico è stato generato prendendo 10 000 punti casuali con numero di ingredienti e passaggi minore di 40.

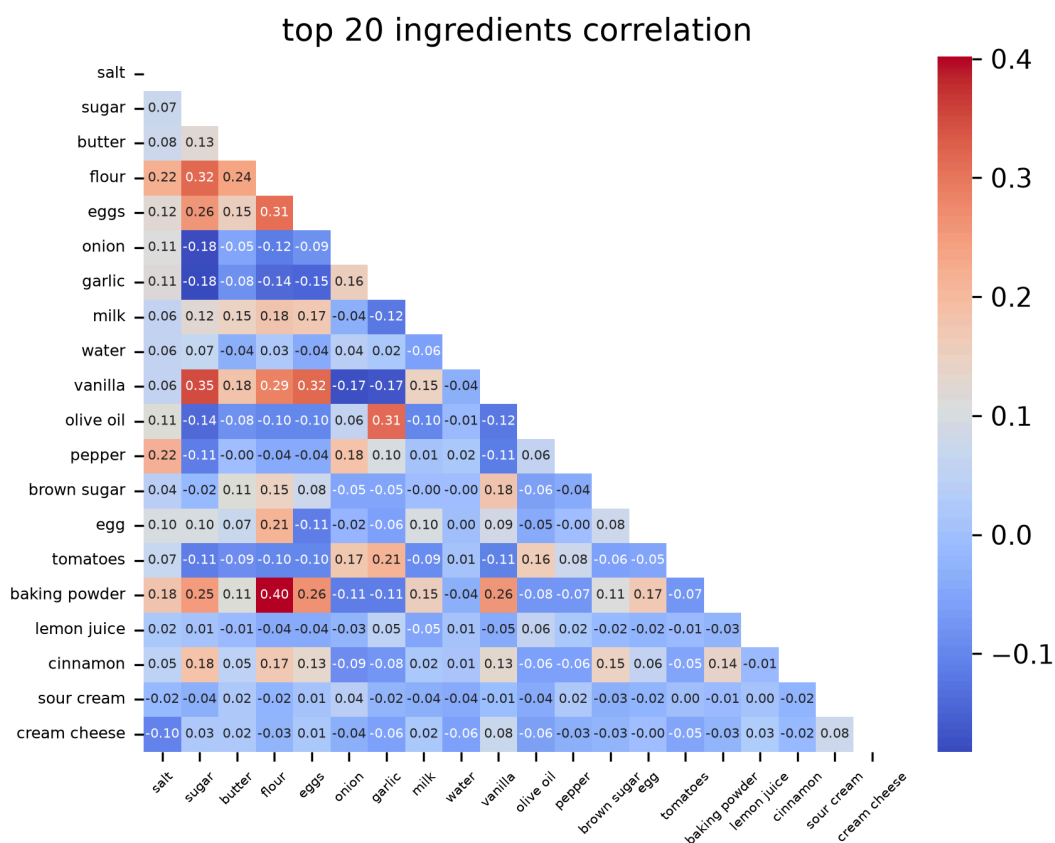
Come possiamo vedere, si ha una grande densità di ricette fino ai 15 ingredienti e passaggi. La relazione tra questi non è tuttavia lineare: due ricette con numero di ingredienti simile possono avere un numero di passaggi diverso tra loro (vale anche il viceversa).

2.9 Ingredienti più frequenti



Come prevedibile, gli ingredienti maggiormente presenti sono quelli di uso comune. In primis troviamo le spezie. Seguono poi gli ingredienti che stanno alla base della maggior parte delle ricette.

2.10 Correlazione tra gli ingredienti



Sopra è mostrata la relazione tra i 20 ingredienti più comuni. Una relazione positiva indica che due ingredienti tendono ad essere insieme in una ricetta. Al contrario, una relazione negativa indica che due ingredienti tendono a evitarsi.

Per esempio, come siamo abituati a pensare, due ingredienti salati tendono a stare nella stessa ricetta mentre uno dolce e uno salato no. Il grafico ci conferma proprio questo: ad esempio farina e lievito compaiono molte volte insieme, mentre aglio e zucchero tendono a evitarsi.

2.11 Conclusioni

Per il nostro sistema di *information retrieval* la colonna più rilevante è la colonna *NER*, che contiene gli ingredienti già normalizzati. Dallo studio del dataset è emerso che questa colonna non presenta grosse criticità, in quanto ha una distribuzione abbastanza compatta e un numero di elementi duplicati limitato. Non sono state quindi necessarie operazioni aggiuntive. L'unica è stata il preprocessing spiegato nel paragrafo 2.2. Lo studio effettuato al di fuori della colonna *NER* non era necessario ai fini del nostro obiettivo. Lo abbiamo comunque lasciato per una eventuale implementazione futura di generative AI.

3. Machine Learning

3.1 Information Retrieval

Come abbiamo già detto, l'obiettivo del progetto è quello di cercare all'interno di un dataset le ricette che sono semanticamente più simili a una query fornita dall'utente.

L'*information retrieval* svolge questo compito ed è l'insieme delle tecniche utilizzate per cercare informazioni rilevanti all'interno di grandi collezioni di documenti. Nel nostro caso i documenti sono rappresentati dalle ricette presenti nel dataset, mentre la query è costituita dall'elenco degli ingredienti inseriti dall'utente.

L'idea di base consiste nel trasformare le ricette e la query in vettori numerici, chiamati *embeddings*, che rappresentano il loro significato semantico. In questo

modo una query e una ricetta simili avranno gli embeddings “vicini” nello spazio vettoriale in cui si trovano.

Il confronto semantico quindi viene fatto prendendo in considerazione la distanza vettoriale tra gli embeddings.

3.2 Sentence-BERT (SBERT)

Per fare la trasformazione in embeddings è stato utilizzato il modello *Sentence-BERT (SBERT)*, una variante di *BERT* più veloce e più efficiente nella ricerca semantica.

Fra le varie scelte, nel progetto è stato usato il modello pre addestrato “*all-MiniLM-L6-v2*” in quanto è fra uno dei modelli più leggeri e veloci, a discapito di ottenere delle prestazioni leggermente peggiori.

3.3 FAISS

Una volta generati gli embeddings questi sono stati salvati all’interno di un indice. Confrontare la query con tutti gli embeddings richiede una ricerca veloce. Per questo motivo è stata utilizzata la libreria *FAISS* che implementa degli algoritmi di ricerca con complessità $O(\log n)$.

3.4 Creazione degli embeddings

Un *embedding* è un vettore numerico di dimensione 1x384. Per la generazione di questi ci siamo trovati di fronte a un problema: uno di noi due aveva a disposizione solo 8GB di RAM e non siamo riusciti a generare gli embeddings in un indice solo. Il progetto è stato quindi portato avanti in due versioni: una per limitare l’utilizzo della memoria a 8GB e una no.

La creazione è stata fatta utilizzando la colonna *NER* del dataset, in quanto la query inserita dall’utente è una lista di ingredienti disponibili.

Prima della creazione sono state fatte delle operazioni di normalizzazione.

Per quanto riguarda la generazione degli embeddings delle ricette:

```
texts = [" ".join(sorted(ingredients)) for ingredients in
dataset["NER"]]
```

Gli ingredienti sono stati quindi ordinati in ordine alfabetico e separati da spazi (durante il preprocessing del dataset erano già stati portati in lower case). Per esempio una ricetta con ["sale", "pepe"] diventa "pepe sale".

Per quanto riguarda la generazione degli embeddings della query:

```
ingredients = " ".join(ingredient.strip().lower() for
ingredient in sorted(ingredients.split(",")))
```

In questo caso gli ingredienti arrivano in input come una stringa che contiene gli ingredienti disponibili separati da virgole. Per prima cosa prendiamo quindi gli ingredienti discriminandoli tra loro quando incontriamo una virgola. Poi li ordiniamo in ordine alfabetico e li portiamo in lower case. Infine costruiamo la stringa degli ingredienti separata da spazi come facevamo per gli embeddings delle ricette. Facendo un esempio, se l'utente inserisce "Sale, Pepe" questa diventa "pepe sale", proprio come prima.

Perché ci interessa questa normalizzazione? Quando andiamo a fare la ricerca semantica utilizziamo la funzione `search()` di FAISS che restituisce uno score di somiglianza tra la query e le ricette che ha trovato. Idealmente vorremmo che se l'utente inserisce "sale pepe" e un embedding della ricetta è stato calcolato sulla stringa "pepe sale" lo score di somiglianza sia massimo, a prescindere dall'ordine degli ingredienti. Questo però di base non succede, perché le due stringhe, seppur hanno lo stesso significato per noi, vengono considerate come diverse. Aggiungendo la parte di normalizzazione risolviamo il problema.

Vediamo adesso le due versioni. La prima è quella senza limitazioni di memoria ed è la più semplice. Per creare gli embeddings infatti basta usare la funzione:

```
embeddings = model.encode(
    texts,
    batch_size=128,
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True
```

)

dove:

- model è il modello “*all-MiniLM-L6-v2*”
- texts sono gli ingredienti normalizzati come detto prima

Gli ultimi due argomenti sono necessari per FAISS perchè la libreria lavora con i numpy array e normalizzare gli embeddings facilita il calcolo della ricerca semantica (paragrafo 3.5). Una volta calcolati, gli embeddings vengono salvati in un indice:

```
index =  
faiss.IndexFlatIP(model.get_embedding_dimension())  
index.add(embeddings)  
faiss.write_index(index, str(INDEX_DIR /  
"embeddings.index"))
```

Si noti che la creazione dell'indice è stata fatta con la funzione `IndexFlatIP()` dove IP sta per *Inner Product*. Grazie alla normalizzazione degli embeddings, FAISS eseguirà la ricerca semantica facendo un prodotto scalare (paragrafo 3.5).

La seconda versione è quella con la limitazione di memoria. Per la creazione degli embeddings usiamo la stessa funzione di prima, ma questa volta `texts` non conterrà le ricette di tutto il dataset. Viene implementata una elaborazione a blocchi del dataset, prendendo in ogni blocco il 10% del totale (223 114 ricette alla volta). Oltre al salvataggio dell'indice degli embeddings, questa volta è necessaria una struttura in più. Facciamo un esempio per spiegarlo. Abbiamo detto che elaboriamo il dataset a blocchi, quindi al primo blocco prendiamo in esame le ricette con indice da 0 a 223 113. Una volta creati gli embeddings questi avranno indice da 0 a 223 113. Fin qui tutto ok. Al secondo blocco prendiamo le ricette con indice da 223 114 a 446 227. Se creiamo gli embeddings questi avranno nuovamente indice da 0 a 223 113. In questo modo non siamo in grado di ricavare la ricetta originale, visto che la ricerca la facciamo sugli indici degli embeddings. Serve quindi un'altra struttura che salvi gli indici globali che prendiamo in esame, in modo da avere un meccanismo di mapping. Riassumendo, nel codice facciamo:

```
np.save(str(INDEX_DIR / f"shard_{shard}.npy"),  
global_indexes)
```

```
faiss.write_index(index, str(INDEX_DIR /  
f"shard_{shard}.index"))
```

dove:

- shard_x.npy contiene gli indici globali che stiamo esaminando
- shard_x.index contiene gli embeddings

3.5 Cosine Similarity

La ricerca semantica avviene calcolando la *cosine similarity* di due embeddings, ovvero viene misurato quanto due vettori puntino nella stessa direzione.

$$\text{CosineSimilarity}(A, B) = (A * B) / (\|A\| * \|B\|)$$

La normalizzazione degli embeddings fa sì che la norma sia unitaria, per cui la cosine similarity si riduce ad un prodotto scalare. Ecco perché viene usato `normalize_embeddings=True` in `encode()` e `IndexFlatIP()` per la creazione dell'indice.

3.6 Recupero delle ricette

Dopo che l'utente ha mandato la query, la ricerca semantica viene fatta nel seguente modo. Prima si creano gli embeddings della query allo stesso modo di come venivano creati quelli delle ricette:

```
embeddings = model.encode(  
    ingredients,  
    convert_to_numpy=True,  
    normalize_embeddings=True  
)
```

Poi viene fatta la ricerca semantica:

```
scores, indexes = index.search(embeddings, top_k)
```

Questa funzione trova le k ricette più simili alla query e restituisce, per ogni ricetta:

- uno score di somiglianza
- l'indice della ricetta trovata (l'indice dell'embedding)

Nella versione senza limitazione di memoria non ci sono problemi, in quanto l'indice dell'embedding corrisponde con l'indice vero della ricetta nel dataset.

Nella versione con limitazione di memoria abbiamo bisogno del mapping:

```
index = faiss.read_index(str(INDEX_DIR /  
f"shard_{shard}.index"))  
global_indexes = np.load(str(INDEX_DIR /  
f"shard_{x}.npy"))  
global_index = global_indexes[local_index]  
recipe = dataset.iloc[global_index]
```

dove `local_index` è l'indice restituito da `index.search()`.

Oltre a questo, abbiamo bisogno anche di una cosa in più. Per trovare le ricette migliori dobbiamo cercare in tutti i singoli blocchi. Infine, dopo aver trovato le migliori k in ogni blocco, è necessario un ordinamento in base allo score per ottenere le migliori k globali.

4. Evaluation

4.1 Metrica di valutazione

La valutazione del modello è stata fatta prendendo in esame la metrica *Recall@N*. Questa tecnica consiste nel trovare la proporzione fra il numero di documenti rilevanti recuperati e il numero di tutti i documenti rilevanti disponibili nella collezione considerata.

Per implementare la valutazione è stato preso un campione casuale di ricette dal dataset. Poi, per ogni ricetta di questo campione, è stato preso un sottoinsieme di ingredienti e dato come input al modello, che ritornerà le N ricette più simili. Il risultato ottenuto viene così confrontato con la ricetta originale per capire se questa è stata trovata e restituita dal modello.

Tradotta al nostro caso d'uso la *Recall@N* è quindi espressa dalla seguente formula:

$$Recall@N = (correct) / (tested)$$

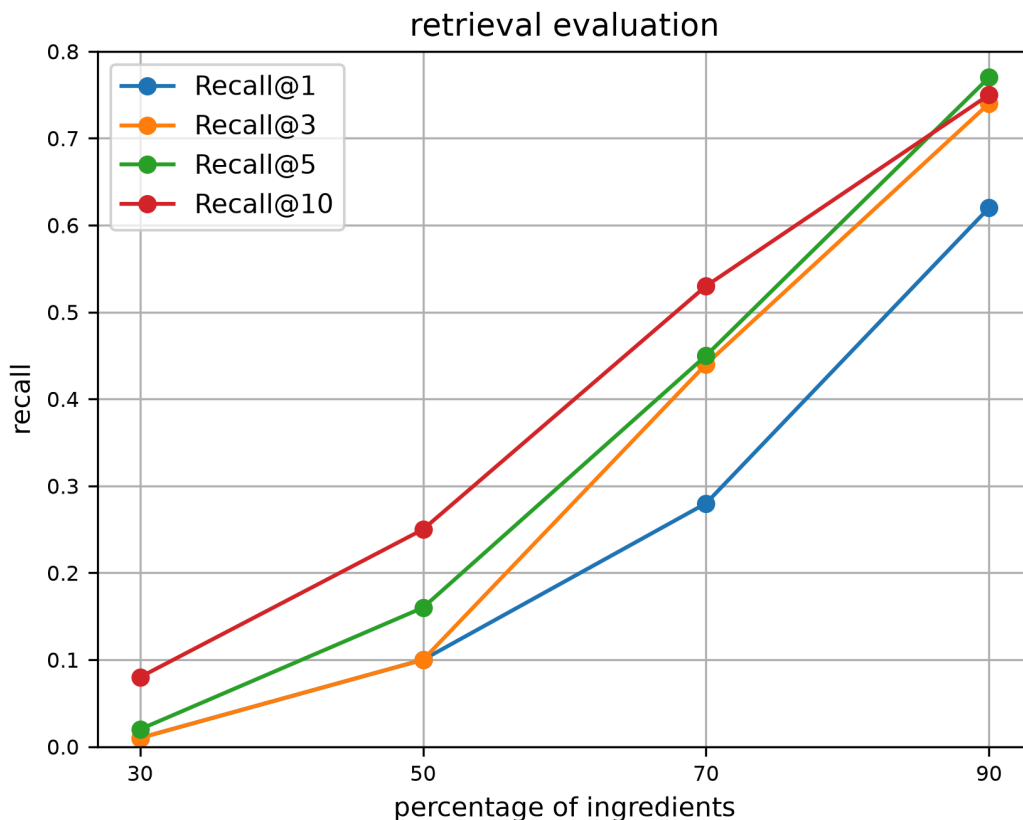
dove:

- *correct* è il numero di volte che una ricetta è stata trovata fra le prime N

- *tested* è il numero di volte che è stato effettuato l'esperimento

4.2 Interpretazione grafica

Il seguente grafico mostra come si comporta il modello al variare del parametro N e della percentuale di ingredienti presa da una ricetta.



Come ci aspettiamo, il grafico si comporta come segue:

- più ingredienti vengono forniti, maggiore è la facilità con cui si recupera una ricetta
- se N aumenta, la probabilità di trovare la ricetta aumenta

Le prestazioni peggiori si hanno ovviamente per il 30% degli ingredienti. Prendendone circa un terzo infatti il modello tende a dare priorità a quelle ricette che hanno numero di ingredienti e ingredienti simili a quel terzo.

Osservando la crescita delle curve ci accorgiamo che:

- se teniamo la percentuale fissa e aumentiamo N , le prestazioni aumentano in media del 15% circa
- se teniamo N fisso e aumentiamo la percentuale, le prestazioni aumentano in media del 68% circa

Quindi in generale è meglio fornire una lista di ingredienti più specifica in quanto il modello si comporta meglio di conseguenza.

5. Interfaccia Web

5.1 Architettura dell'applicazione

Per rendere il sistema facilmente utilizzabile è stata sviluppata una applicazione web che permette all'utente di cercare ricette direttamente dal browser.

L'applicazione è stata realizzata usando *Flask*, e si basa su una architettura *client-server*. Il client è rappresentato dal *frontend*, che si occupa principalmente di interagire con l'utente e contattare il server per fargli una richiesta. Il server è rappresentato dal *backend* e si occupa di eseguire la ricerca semantica a seguito di una richiesta del client.

5.2 Frontend

Il *frontend* espone due pagine:

- la prima permette all'utente di inserire gli ingredienti per fare la richiesta
- la seconda viene mostrata a seguito della richiesta e visualizza il risultato della ricerca semantica

Quando l'utente inserisce gli ingredienti, il frontend effettua una richiesta HTTP di tipo POST verso il backend e invia gli ingredienti inseriti dall'utente in formato JSON.

5.3 Backend

Il *backend* carica all'avvio il dataset, il modello e gli embeddings e li tiene in memoria finché resta attivo. In questo modo velocizziamo il più possibile le richieste, introducendo overhead solo per la ricerca semantica.

All'arrivo di una richiesta il backend esegue la ricerca semantica e restituisce i risultati al frontend in formato JSON.

6. Docker

Per rendere l'applicazione portatile e standardizzare l'ambiente di esecuzione è stata implementata la dockerizzazione tramite due Dockerfile e la suddivisione in due servizi, Backend e Frontend, tramite docker-compose.

Inoltre per rendere l'immagine il più leggera possibile è stato aggiunto un file .dockerignore per limitare la copia ai soli file necessari al funzionamento dell'applicazione.

6.1 Dockerfile

Dockerfile del backend:

```
FROM python:3.12-slim

WORKDIR /app

COPY app/backend/requirements.txt .
RUN pip install --no-cache-dir --default-timeout=1000 \
    --extra-index-url
    https://download.pytorch.org/whl/cpu \
    -r requirements.txt

COPY config.py .
COPY app/backend/ ./app/backend/
```

```
WORKDIR /app/app/backend
```

```
CMD ["python", "api.py"]
```

Dockerfile del frontend:

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY app/frontend/requirements.txt .
```

```
RUN pip install --no-cache-dir --default-timeout=1000 -r  
requirements.txt
```

```
COPY app/frontend/ ./app/frontend/
```

```
WORKDIR /app/app/frontend
```

```
CMD ["python", "app.py"]
```

I *dockerfile* presentano una struttura semplice e il loro scopo principale è installare tutte le dipendenze e i moduli necessari al funzionamento dell'applicazione. Per questo motivo sono stati utilizzati due file *requirements.txt* diversi. Notare che nel dockerfile del backend è presente l'opzione:

```
--extra-index-url https://download.pytorch.org/whl/cpu
```

che serve per installare le dipendenze *cpu-only* ed evitare quelle per l'accelerazione della GPU, che non abbiamo usato.

L'immagine di base scelta è la versione ufficiale *slim* di python 3.12 in modo da ridurre la dimensione dell'immagine utilizzando solo i pacchetti di python strettamente necessari.

6.2 Docker-compose

La gestione della comunicazione tra i due diversi servizi offerti dall'applicazione è stata implementata tramite un file *docker-compose.yml*.

Questo file definisce, per ogni servizio (frontend e backend) le rispettive workdir, le porte su cui verranno esposti i servizi e l'implementazione di una rete virtuale dedicata alla comunicazione con driver bridge.

Inoltre nel servizio backend viene configurata la variabile d'ambiente PYTHONPATH per garantire la corretta risoluzione dei moduli interni usati per la ricerca delle ricette. Il dataset e gli embedding vengono inseriti nell'immagine come volumi, in particolare vengono creati due volumes nel backend per il dataset e per idx che contiene gli embeddings.

Le porte esposte da backend e frontend sono rispettivamente la 8000 e la 5000 che vengono mappate nella macchina host in 8080 e 5050.